

Authenticated but Unauthorized: Bounded-Latency 5D H-NGAC for Real-Time Robotics

Hassan Karim*, Md Omar Faruque[†], Abdel-Hameed A. Badawy[‡], Sai Sitharaman[§], Deepti Gupta[¶]

*Stable Cyber LLC [†]New Mexico State University [‡]Florida International University

[§]Zetafence, Inc. [¶]Texas A&M University - Central Texas

Abstract—Real-time robotic and drone system designers often leave authorization out of the control path because software policy engines do not offer decisions within a fixed cycle budget. Operating system scheduling, not policy evaluation, creates non-deterministic latency, so a faster software-based engine does not offer the guarantees required by hard real-time applications. The gap leaves attack classes open that identity-based authorization cannot see, including unsafe-state operation and command injection from compromised but credentialed nodes. State of the art H-NGAC compiles NGAC policy graphs into fixed-width hardware bitmasks so that an authorization decision reduces to a chain of bitwise AND operations. In this paper, we improve hardware security controls by extending H-NGAC with two further dimensions, runtime system state and command provenance. We demonstrate this with three-, four- and five-dimensional kernels on a Zynq-7020 FPGA. We also evaluate the five-dimensional policy in an adversarial Robot Operating System 2 (ROS 2) scenario against a compromised node that has already authenticated, where it blocked 18,878 of 18,878 injected commands and passed 17,059 of 17,059 legitimate ones. We found that the added security dimensions are free in time. All three kernels resolve in an identical number of clock cycles at every policy size tested, with a closed-form worst case and zero jitter by construction. On the same corpus the three-dimensional kernel permits every request the five-dimensional kernel denies. In software, those same dimensions make every policy rule more expensive, the fifth by 19%. The two added dimensions cost 1,336 LUTs and no BRAM or DSP. With 5D H-NGAC, robotics and functional-safety engineers can now add a security dimension to the control path without spending clock cycles.

Index Terms—access control, NGAC, FPGA, high-level synthesis, real-time systems, robotics, ROS 2, worst-case execution time, provenance, cyber-physical security

I. INTRODUCTION

A quadruped inspection robot in maintenance mode should not move. Layla, a field robotics engineer, is replacing a battery pack on a 45 kg rover when an “increase velocity” command arrives from a remote operator console. The Data Distribution Service (DDS) transport authenticates the console’s certificate and delivers it in well under a millisecond. Nothing asks whether a remote operator may command motion while a human is at the rover and the platform is locked for maintenance. If the rover acts, Layla could be hurt. Figure 1 shows the path. CVE-2022-33323 [1] demonstrates this failure mode against Mitsubishi MELFA industrial robots, and CVE-2021-38425 [2] documents message injection against eProsima Fast DDS, the default ROS 2 transport.

Robotics platforms leave authorization out of the control path for a defensible reason. Although modern IT enterprise

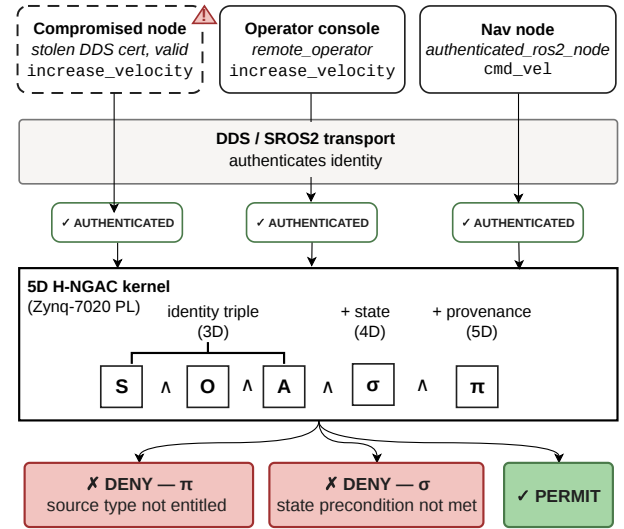


Fig. 1. Authenticated but unauthorized. Three commands clear DDS/SROS2 authentication and arrive as equals. The five-dimensional kernel separates them, one on the state precondition the maintenance lock clears and one on provenance. A three-dimensional kernel permits all three.

policy engines evaluate rich attribute-based policy, they cannot guarantee decision timing. That is, the latency cost of a software policy decision point is governed by operating system scheduling, preemption and cache state, not by policy evaluation, and thus offers no bounds by which to plan. So, a faster engine may not offer a tighter bound. In previous measurements [3], an OPA edge deployment took 1 to 5 ms per decision and an XACML decision point near 50 ms, while a software authorization path averaging 1.14 μ s still produced a single 157 μ s scheduling outlier. A further evaluation [4] measured OPA at a 271.491 μ s mean with 13 missed real-time deadlines. A control loop cannot budget for a tail it cannot bound. It is for this reason that robotic system designers place authorization at the perimeter and leave the control path open.

Our research approaches this as a compilation problem. The hypergraph privilege analysis in [5] established that large privilege structures compile into forms that answer set-level questions quickly. H-NGAC [3], presented at IEEE DCAS 2026, applied that idea to the decision path timing. By compiling the NGAC policy graph [6], [7] into fixed-width bitmasks, they reduced one authorization decision to a chain

of bitwise AND operations. H-NGAC, however, only checked the identity triple of subject, object and attribute. It did not model the platform’s runtime state, and it did not model where a command came from. Layla’s maintenance lock flag and a remote operator’s increase-velocity problem exist within the missing dimensions.

In this paper we extend H-NGAC with those two dimensions and synthesize the result to reconfigurable logic. We contribute three things:

- 5D H-NGAC, an authorization model that adds a system-state mask σ and a command-provenance mask π to the H-NGAC identity triple.
- A synthesized hardware evaluation on a Zynq-7020 showing three-, four- and five-dimensional kernels with equal clock cycles at every policy size tested.
- An adversarial ROS 2 evaluation against five-dimensional policy and a six-model software comparison.

The remainder of this paper is organized as follows. Section II scores related work. Section III gives the threat model. Section IV defines the 5D decision function and Section V renders it into hardware. Section VI covers method, Section VII results, Section VIII limitations, and Section IX concludes.

II. RELATED WORK

State-of-the-art work approaches these problems from different angles: per-decision latency, worst-case boundedness, dimensionality, and hardware validation. To demonstrate novelty, we score prior work on these axes, each an integer $x \in \mathbb{Z}$, $0 \leq x \leq 5$. Per-decision latency evidence (*pdl*): 5 requires cycle-accurate per-decision measurement. Worst-case boundedness (*wcb*): 5 requires a closed-form bound verified at every measured point. Dimensionality beyond the identity triple (*dim*): 3 is one added runtime dimension, 5 is both state and provenance. Hardware validation (*hvv*): 5 requires cycle-accurate co-simulation plus silicon verification. Table I applies these definitions.

Software policy engines and models. The NGAC standard [6] and the Policy Machine [7] define the model we compile, but report no per-decision latency. NGAC offers decision and review advantages over XACML [26], and to our knowledge every NGAC implementation to date is software, including NIST’s fine-grain enforcement over database queries [27]. The strongest software decision result is the linear-time graph algorithm [8]. But that is still a per-decision traversal with a constant that depends on the host. XEngine [9] compiles XACML into numeric structures offline, but its decision latency still varies with policy and request shape. Reported measurements like [4] place OPA [10] and an XACML PDP [11] three to five orders of magnitude above the bound derived in Section V.

Hardware enforcement. Three lines of work approach authorization in hardware. Formally specified memory-isolation policy was compiled into an FPGA reference monitor in 2006 [12]. Access-rights checks were put into NoC interfaces with no added latency on the authorized path [13]. Although both make enforcement latency a property of a synthesized

TABLE I
RELATED WORK SCORED ON THE FOUR RUBRIC AXES. CELL SHADING TRACKS THE SCORE, LIGHTEST AT 0 AND DARKEST AT 5, SO THE ORDERING READS IN GRAYSCALE.

Prior work	<i>pdl</i>	<i>wcb</i>	<i>dim</i>	<i>hvv</i>
<i>Software policy engines and models</i>				
NGAC standard / Policy Machine [6], [7]	1	0	2	0
Linear-time NGAC [8]	3	1	2	0
XEngine [9]	3	0	3	0
OPA [10]	3	0	3	0
XACML PDP [11]	2	0	3	0
<i>Hardware enforcement</i>				
FPGA reference monitor [12]	3	3	1	4
NoC data protection units [13]	3	3	1	3
Bit-vector classification [14], [15]	4	4	0	4
SoC access-control wrappers [16]	3	3	1	4
CHERI [17]	4	4	0	5
<i>Real-time and robot security</i>				
SROS2 / DDS-Security [18], [19]	2	0	0	0
App-layer ROS authorization [20]	2	0	1	0
ROS-Defender [21]	2	0	1	0
ABAC for multi-robot systems [22]	2	0	3	0
Security-aware RT scheduling [23], [24]	3	2	1	0
CPS runtime enforcement [25]	2	3	1	0
<i>Prior NGAC work</i>				
HyperNGAC [5]	0	0	3	0
H-NGAC [3]	4	2	0	1
TS-NGAC [4]	4	3	3	0
5D H-NGAC (this work)	5	5	5	5

circuit, neither evaluates a relational policy model nor prices an added policy dimension. The third is packet classification. TCAM and bit-vector engines match wide multi-field keys at deterministic rates [14], and StrideBV [15] advertises latency independent of ruleset content. However, StrideBV latency grows linearly with match-key width, constraining its generalizability. In the closest deterministic prior art, added dimensions are not free in time, unlike ours. The Aker model [16] wraps SoC controllers in verified access-control hardware at the bus-transaction level, and CHERI [17] enforces per-instruction memory capabilities in silicon. Both police a lower layer than a policy model.

Real-time and robot security. This literature schedules software enforcement around deadlines. Contego slots security tasks into scheduling slack [23]. A 2024 SoK [24] catalogs scheduling in robot security controls but misses authorization completely. We leave that paradigm rather than optimize within it, because a fabric decision consumes no CPU slack. Runtime enforcers were synthesized in [25] from timed automata for reactive CPS, the nearest formal-synthesis neighbor to this work. It targets safety properties rather than authorization, and offers no hardware latency model. In robotics, [20] argued for application-layer authorization above transport crypto, ROS-Defender filters node flows at the network layer [21], and [22] conditions ABAC decisions on task and capability attributes through a blockchain bridge. None grants command permission based on its source type or the platform’s runtime state, and none is an inline bounded-latency primitive.

This work scores the maximum on all four axes, and Sec-

tion VII supplies the evidence for that row. It measures cycle-accurate per-decision latency, verifies a closed-form bound at every measured point, enforces both state and provenance, and validates by co-simulation, silicon functional verification, and a hardware-latched on-board cycle capture that tracks co-simulation to a constant offset. The novelty claim is bounded. Constant-latency wide-key matching exists in TCAMs, deterministic classification exists in the bit-vector family, and policy-to-circuit compilation dates to 2006. To our knowledge, no prior work compiles a standardized relational authorization model to hardware and shows, on one toolflow and part, that added policy dimensions cost zero clock cycles, and no hardware implementation of NGAC exists at all.

III. BACKGROUND AND THREAT MODEL

A. From DAG traversal to bitmask evaluation

NGAC, standardized as INCITS 565, expresses policy as a directed graph of users, objects and attributes, and answers a query by reachability traversal [6], [7]. That generality costs a traversal on every decision. H-NGAC compiles the graph ahead of time into fixed-width bitmasks, so the runtime decision is a chain of AND operations over precomputed sets [3]. We call the traversal-based reference model DAG-NGAC. This paper extends H-NGAC and inherits its compilation step unchanged.

B. Attack classes

We consider three named attack classes, each anchored to a published CVE against deployed industrial or robotic systems.

Unauthorized access. The attacker acts as an unprivileged agent or through a hijacked session (CVE-2022-45789, Schneider Electric Modicon [28]). This class of threats is adequately addressed by traditional identity-triple access control models.

Unsafe-state operation. A malicious or confused client issues a well-formed command that the platform's current state does not license. The same drive command is routine while the rover is operating and hazardous once a technician locks it for maintenance (CVE-2022-33323, Mitsubishi MELFA [1]). The drive rule names the operating state as a precondition, and the maintenance lock clears it, so the command permitted a moment earlier is denied. Identity-based authorization admits this class because the request is legitimate on every credential it carries.

Command provenance abuse. The attacker holds valid DDS credentials for an authorized subject but is not an entitled source type for the command it issues (CVE-2021-38425, eProxima Fast DDS [2]). SROS2 and DDS-Security authenticate who a node is at the transport layer [18], [19], but never ask whether that source type may issue that command to that resource. By design that question is invisible below the application layer. Further, transport access control is itself fragile as [29] demonstrated giving four ways a credentialed insider bypasses SROS2 permissions and keeps publishing.

A fourth concern (not an attack class) is the timing window, in which a correct DENY arrives after the actuator has acted.

This spans all three classes. It is only closed by bounding decision latency below message propagation time, not by adding more policy clauses.

C. Assumptions

We assume the attacker holds valid transport credentials for at least one authorized subject and can publish at wire rate. Transport security is intact, and we do not address broken cryptography. The decision point and the compiled policy are trusted. Compilation happens off the decision path, and policy updates are outside our present scope. The platform exposes its runtime state to the decision point as a bit vector.

IV. THE 5D H-NGAC MODEL

We formalize the decision function before describing hardware. Let U be the universe of policy nodes with $|U| = 256$. A set over U is a bitmask $M \in \{0, 1\}^{256}$, stored as four 64-bit words.

Definition 1 (Policy rule). A policy rule is a five-tuple

$$r_i = (S_i, O_i, A_i, \sigma_i, \pi_i) \quad (1)$$

where $S_i, O_i, A_i \in \{0, 1\}^{256}$ are the permitted subject, object and attribute sets, $\sigma_i \in \{0, 1\}^{32}$ enumerates the state bits that must be asserted for the rule to apply, and $\pi_i \in \{0, 1\}^{32}$ enumerates the source types entitled to invoke it. A zero σ_i or π_i is a wildcard.

Definition 2 (Request). A request is a five-tuple $q = (s, o, a, \sigma_q, \pi_q)$ with subject identifier $s \in \mathbb{Z}$, $0 \leq s < 256$, object identifier $o \in \mathbb{Z}$, $0 \leq o < 256$, required attributes $a \in \{0, 1\}^{256}$, the platform's current state vector $\sigma_q \in \{0, 1\}^{32}$, and the requester's provenance $\pi_q \in \{0, 1\}^{32}$.

Definition 3 (Match and decision). Rule r_i matches request q when

$$\begin{aligned} \text{match}_i(q) &= S_i[s] \wedge O_i[o] \wedge (a \subseteq A_i) \\ &\wedge (\sigma_i \subseteq \sigma_q) \wedge (\pi_i = 0 \vee \pi_i \cap \pi_q \neq \emptyset) \end{aligned} \quad (2)$$

and the decision over a policy of n rules is

$$\delta(q) = \bigvee_{i=0}^{n-1} \text{match}_i(q), \quad \delta \in \{0, 1\}. \quad (3)$$

Every term in (2) is a bitwise AND against a precompiled mask followed by a comparison. The first three terms are the H-NGAC identity triple. The σ term conditions the permission on the platform's runtime state, and the π term conditions it on the command's source type. Dimensions four and five add two AND terms to a conjunction that already exists. They add no traversal, no lookup and no branch. One convention governs σ . A rule names the states in which it may fire, so the encoding is positive and monotone. Asserting a state bit can only make more rules match, so the model denies on a missing precondition, not on a hazard flag being set. On scope, we compile only the association subset of INCITS 565, that is, the rules that grant. Prohibitions, obligations and administrative relations are not compiled, and Section VIII bounds our claims accordingly.

Hypothesis. An added security dimension is free in time if and only if its evaluation synthesizes to additional combinational inputs within an existing pipeline stage, rather than to an additional sequential stage or memory access. Equation (2) makes σ and π satisfy this condition. Each is one AND-and-compare against a 32-bit register, far narrower than the 256-bit identity terms beside it. Section VII tests this directly.

Scaling in n separates the two implementations. As n grows toward the 512-rule capacity, hardware cost grows only through scan length, and Section V shows that growth is half a cycle per rule regardless of how many dimensions a rule carries. In software the growth compounds with dimensionality: our measured per-rule slope rises from 0.645 cycles in three dimensions to 1.220 in five. The two curves diverge as policy gets richer.

V. HARDWARE ARCHITECTURE

The kernel evaluates (3) as a pipelined linear scan. Each clock, a two-stage pipeline loads the next two rules from policy memory while evaluating the two rules loaded on the previous clock, giving 0.5 cycles per rule at an initiation interval of one. The policy sits behind a BRAM-style port and the request and rule count arrive over AXI-Lite registers. Synthesis maps the evaluation entirely to LUTs and flip-flops, and the kernel consumes zero BRAM and zero DSP blocks.

Figure 2 shows the system as built for the board measurement of Section VII-F. Policy memory is a dual-port block RAM, so the processing system can rewrite policy on one port while the scan reads the other, keeping update traffic off the decision path. The AXI Timer is instrumentation, not part of the primitive. It latches the decision interval in hardware, which is what makes the board counts in Table V measurements of the fabric rather than of its caller. The constant 25-cycle offset those counts carry is the round trip through the interconnect.

Determinism is structural, not observed. The scan executes $\lceil n/2 \rceil + 1$ pipeline iterations regardless of where, or whether, a match occurs. A permit latches the decision but does not shorten the scan, and an odd rule count pads with an all-zero rule that cannot match. Thus minimum, mean and maximum latency are equal by construction, and the measured closed form

$$L(n) = 12 + n/2 \text{ cycles} \quad (4)$$

holds exactly at every policy size we tested, where the constant 12 covers interface handshake and pipeline fill. At 100 MHz, $L(500) = 262$ cycles is $2.62 \mu\text{s}$, and the 512-rule capacity bounds the primitive at $2.68 \mu\text{s}$. A designer needing a tighter bound shrinks n or raises the clock. Either way the bound is known before deployment, which is what makes the worst case a synthesis parameter.

Forgoing early exit costs the average case. A software scan that exits on first match beats (4) whenever the match lands early. We spend the average case to buy a worst case with zero variance, because a control loop budgets against the worst case.

VI. EXPERIMENTAL METHOD

A. Platforms

Hardware results come from Vitis HLS 2025.2 targeting the Zynq-7020 (xc7z020-clg400-1, the PYNQ-Z1 part) at a 10 ns clock. Three kernels were synthesized and co-simulated: three-dimensional checking the identity triple, four-dimensional adding runtime state, and five-dimensional adding command provenance. They differ only in the conjunction terms of the rule check. Policy layout, pipeline structure, interface and testbench are common. Cycle counts are taken from Verilog co-simulation transaction reports, which are cycle accurate, and are corroborated on PYNQ-Z1 silicon by an AXI Timer that latches the decision interval in hardware. Functional correctness is verified on the same silicon. Software results come from an Intel i7-12800H running the same kernel code compiled with g++ -O3, measured with Linux perf over 200,000 iterations after 1,000 warmup iterations per model. The ROS 2 evaluation runs on ROS 2 Jazzy.

B. Request corpus

For each policy rule the generator emits one request that satisfies all five dimensions, plus a state-failing and a provenance-failing variant where applicable. That yields eleven requests at four rules and 1,334 at 500, or 2,307 across all six sizes. The corpus exercises permits, state denials and provenance denials at every policy size, and the correctness columns of Section VII are computed on it. RTL results use this corpus, sized to co-simulation runtime. The software comparison of Section VII instead replays a request mix over 200,000 iterations, so its denominators are decision counts rather than distinct requests.

C. Honesty rules

We label every number by how it was obtained. Hardware cycle counts are measured and deterministic, and no confidence interval applies because variance is zero by construction. Software per-decision cycle counts are derived, not counter reads: perf supplies a measured effective clock of 4.96 GHz, and cycles are mean nanoseconds times that clock. Proportions from the adversarial run carry exact denominators and one-sided 95% Wilson bounds. One baseline, RBAC with a modeled external state lookup, is a synthetic busy-wait, and we exclude it from every empirical claim in favor of the measured SQLite variant. The timing-window measurement produced zero slips and is reported only as a null result. Board cycle counts are hardware-latched by an AXI Timer and are measurements. Board round-trip latency includes the Linux userspace stack on the processing system and is reported as an observation of that stack, never as a property of the kernel.

VII. RESULTS

A. The key finding: dimensionality is free in time

The cycle column of Table II reports co-simulated kernel cycles per decision. The three-, four- and five-dimensional kernels resolve in an identical number of cycles at every policy

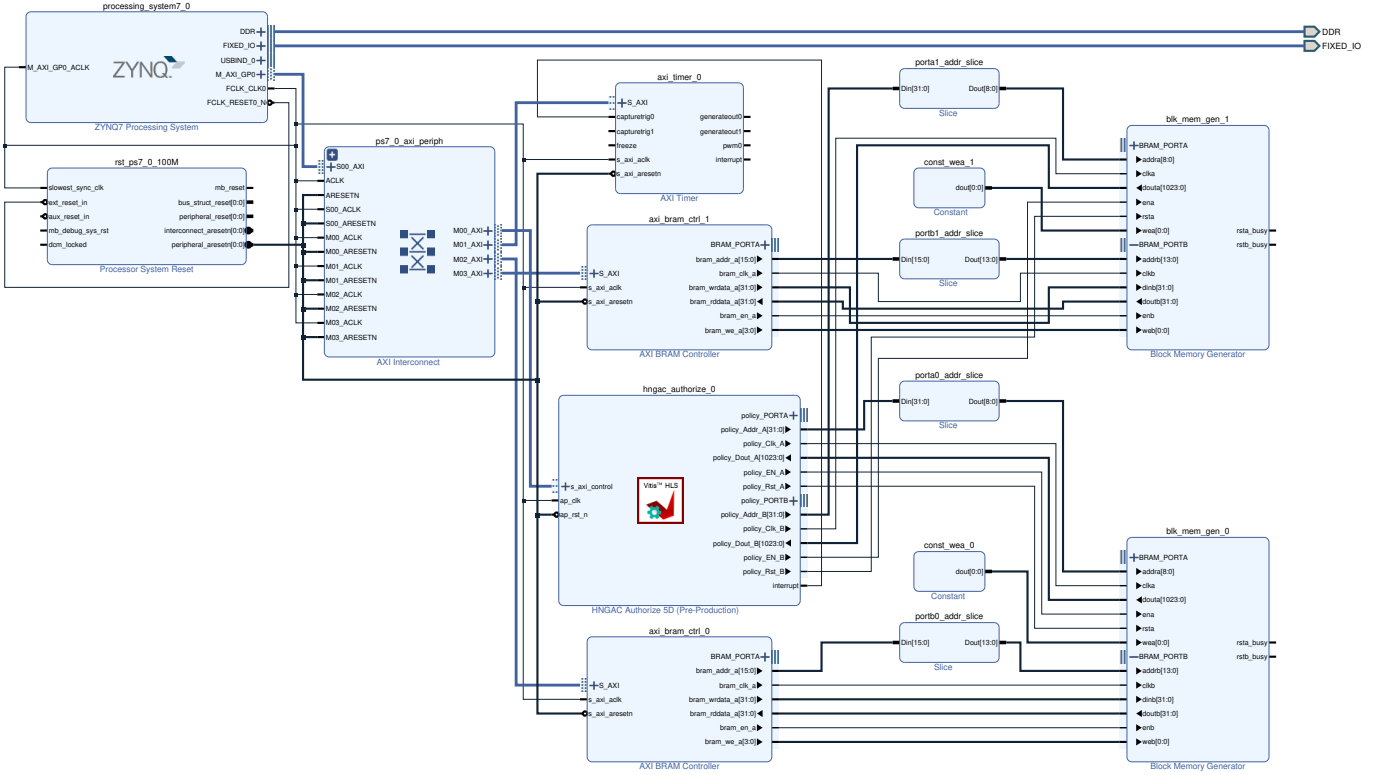


Fig. 2. Vivado block design of the measured system on the PYNQ-Z1. The five-dimensional kernel (hngac_authorize_0, centre) reads policy from dual-port block RAM through two AXI BRAM controllers and receives requests from the Zynq processing system over the AXI interconnect. The AXI Timer (upper centre) latches the decision interval in hardware and is the source of the board cycle counts in Table V.

TABLE II
SAME CYCLES, DIFFERENT DECISIONS. ONE CORPUS, THREE CO-SIMULATED KERNELS. CYCLES PER DECISION ARE IDENTICAL ACROSS 3D, 4D AND 5D AT EVERY POLICY SIZE, WITH MINIMUM, MEAN AND MAXIMUM EQUAL. THE PERMIT COLUMNS ARE NOT IDENTICAL. THE 3D COLUMN SIMPLY REPEATS THE REQUEST COUNT, BECAUSE IT DENIES NOTHING.

Rules	Requests	Cycles	Requests permitted		
		3D=4D=5D	3D	4D	5D
4	11	14	11	8	4
10	27	17	27	19	10
50	134	37	134	92	50
100	267	62	267	184	100
200	534	112	534	367	200
500	1,334	262	1,334	917	500
Total	2,307	—	2,307	1,587	864

size, with identical initiation interval, an identical 6.965 ns estimated clock against the 10.00 ns target, and identical 0.33 ns timing slack. Minimum, mean and maximum are equal at every point, all three co-simulations pass, and the closed form (4) fits all eighteen points exactly. Adding the state dimension to the identity triple costs zero clock cycles, and adding provenance on top of state costs zero more.

One row of Table III is not identical. Iteration latency is two cycles in three dimensions and three in four and five, so pipeline depth grows by one stage when state arrives and

then stops when provenance is added. Because the initiation interval is one, that depth is amortized across the scan and never reaches the per-decision cycle count. That is why the cycle column of Table II is flat while pipeline depth is not. Dimensionality is absorbed into the width of a pipeline stage, not the length of the pipeline, until a stage runs out of timing slack.

Dimensionality is not free in area. Table III prices it. The fourth dimension costs 812 LUTs (+21.6%) over the identity triple and the fifth a further 524 (+11.4%), so carrying both costs 1,336 LUTs (+35.5%) and 331 flip-flops (+14.1%). All three kernels use zero BRAM and zero DSP. The five-dimensional kernel occupies 9% of the part against the three-dimensional kernel's 7%, so two added dimensions cost about two percentage points of a \$100 device. An earlier four-dimensional build completed Vivado place and route with +2.170 ns worst negative slack and zero failing endpoints of 1,987, so the co-simulated clock is not optimistic for this fabric.

B. Identical cycles, different security

Table II puts both halves of the result side by side. One corpus of 2,307 requests went through all three co-simulations, so every number in it is a decision taken by RTL rather than by a software model of it. Read the cycle column and the three kernels are one number. Read the permit columns and they are three.

TABLE III
RESOURCE UTILIZATION FROM SYNTHESIS (ZYNQ-7020, 100 MHz).
PERCENTAGES ARE OF THE AVAILABLE PART RESOURCE.

Metric	3D	4D	5D
II	1	1	1
Iteration latency	2	3	3
LUT	3,768 (7%)	4,580 (8%)	5,104 (9%)
FF	2,348 (2%)	2,579 (2%)	2,679 (2%)
BRAM	0	0	0
DSP	0	0	0
Est. clock	6.965 ns	6.965 ns	6.965 ns
Timing slack	0.33 ns	0.33 ns	0.33 ns

The three-dimensional kernel permits the entire corpus, all 2,307 requests, at every policy size. This is not a defect. Every request carries a valid identity triple and the variants differ only in the state and provenance fields, so a kernel that cannot represent σ or π has no basis to deny any of them. It answers its own question correctly. That question is not the one Layla’s interlock asks.

The four-dimensional kernel denies the state-failing variants and admits the provenance-failing ones, permitting 1,587. The five-dimensional kernel permits 864, exactly one satisfying request per rule at each policy size. Across the total row, 1,443 requests separate three dimensions from five, and 723 of those separate four from five. Each gap is an attack class: unsafe-state operation (CVE-2022-33323) and command provenance abuse (CVE-2021-38425).

The two halves move independently. That is the finding. In hardware, the security question a kernel can ask is decoupled from what asking it costs in time.

C. In software, the same dimension is not free

The same kernel logic compiled for the i7 and measured under perf tells the opposite story. Per-rule marginal cost rises from 0.645 cycles in three dimensions to 1.024 in four and 1.220 in five, a 19% increase for the fifth dimension alone. In software every rule pays for every dimension on every decision, and the cost compounds with policy size. At 500 rules the five-dimensional software path spends 685 derived cycles per decision against the hardware kernel’s 262 measured cycles. Figure 3 plots both families. The software curves fan out as dimensions are added, and the three hardware curves are one line.

D. Faster baselines are faster because they are wrong

Table IV compares six software models over 200,000 replayed decisions. The RBAC hash map resolves in roughly 100 derived cycles at every policy size, and that flat cost is genuine. It also admits 200,000 of 200,000 decisions, including all 100,000 that violate state, and the DAG-NGAC and three-dimensional paths admit the same. A model that cannot see state or provenance cannot deny on them. The only software models that decide correctly are the five-dimensional path and a flattened direct-lookup table that buys its speed with $19.7\times$ the memory. An RBAC deployment could encode state into its

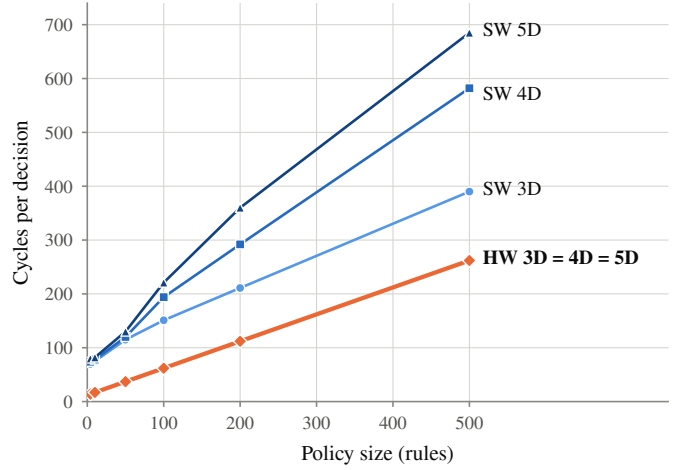


Fig. 3. Cycles per decision versus policy size. Software cycles are derived from perf-measured means and the 4.96 GHz clock; hardware cycles are co-simulated. Software slopes steepen with each added dimension. The 3D, 4D and 5D hardware series are identical at every measured point and render as one line.

TABLE IV
SOFTWARE MODELS, DERIVED CYCLES PER DECISION AT 500 RULES, AND CORRECTNESS OVER THE REPLAYED REQUEST MIX. “OVER” = ADMITS STATE- OR PROVENANCE-VIOLATING REQUESTS.

Model	Cycles (500)	Decides correctly?
H-NGAC 3D	390	No, over-authorizes
H-NGAC 4D	582	No, over-authorizes
H-NGAC 5D	685	Yes
RBAC hash map	100	No, over-authorizes
DAG-NGAC traversal	1,032	No, over-authorizes
Flattened 5D lookup	984	Yes, $19.7\times$ memory

roles, a maintenance operator distinct from a normal operator, but sixteen state bits and three source types multiply that role count combinatorially. The alternative, RBAC consulting an external state store, cost 20.6 times the four-dimensional path against an in-process SQLite store. Table IV is not an argument that cheap static authorization suffices. The fast rows are fast because they answer a smaller question than the one Layla’s interlock asks.

E. Adversarial evaluation in ROS 2

We ran the provenance-abuse scenario live. A software gatekeeper node running the same five-dimensional decision function fronted an actuator topic while a legitimate node and a compromised node both published with valid credentials for the same subject, the compromised node at five times the legitimate rate. Over a 30-second session the policy blocked 18,878 of 18,878 injection attempts, a 100% block rate with a one-sided 95% Wilson lower bound of 99.99%, and passed 17,059 of 17,059 legitimate commands, a 0% false positive rate with a 95% upper bound of 0.02%. This run and the silicon run carry separate claims. The first establishes what the five-dimensional semantics block under sustained adversarial load, the second that the synthesized kernel reproduces those

TABLE V
ON-BOARD AXI TIMER CAPTURE (PYNQ-Z1, 100 MHZ, FULL CORPUS)
AGAINST CO-SIMULATED KERNEL CYCLES. BOARD MINIMUM, MEAN AND
MAXIMUM ARE EQUAL AT EVERY POLICY SIZE.

Rules	Co-sim cycles	Board cycles	Offset
4	14	39	25
10	17	42	25
50	37	62	25
100	62	87	25
200	112	137	25
500	262	287	25

decisions on all 2,307 corpus requests. Under three- or four-dimensional policy every injection would have been admitted, because the attacker’s identity triple and state context were valid. Only provenance separates the two streams.

The same runs measured the timing window. Under idle and saturated CPU load, zero of 8,733 authorization callbacks exceeded the 50 μs lower bound of DDS localhost propagation. We report this as a null result. The argument stands on (4): a synthesis-time bound below the propagation floor closes the window by construction, which no software mean can do.

F. On silicon

Co-simulation is cycle accurate but still simulation. We placed the five-dimensional kernel on a PYNQ-Z1 at 100 MHz behind an AXI Timer and read the decision interval from a hardware-latched counter. Table V reports it against the co-simulated counts.

The board tracks co-simulation with a constant 25-cycle offset at all six policy sizes, and minimum, mean and maximum are equal on silicon as they are in simulation. The offset is the AXI-Lite handshake that moves the request in and the decision out. Being constant, it enters (4) as an additive term rather than a source of variance. The determinism argued from pipeline structure in Section V holds on the real part.

The same run measured round-trip latency from the processing system, and that number behaves differently. Mean round trip rises from 2.09 μs at four rules to 4.77 μs at 500, but the maximum reaches 5.92 ms at 500 rules, three orders of magnitude above its own mean. The tail comes entirely from the Linux userspace stack on the board’s Cortex-A9 between the timer read and the application, the same mechanism behind the i7’s tail in Section VII-G. A deterministic primitive does not make a non-deterministic caller deterministic. It bounds the part of the path a designer can bound, and we present that in Table V.

G. Mean versus worst case

The fabric does not outpace the CPU, and we say so directly. At 500 rules the i7 completes a mean five-dimensional decision in 138 ns of wall clock against the fabric’s 2.62 μs , because a 4.96 GHz core is 49.6 times faster per cycle than a 100 MHz fabric. The CPU wins the mean by roughly 19 $\times$. The defensible comparison is the worst case, and there the ordering reverses. The software path’s worst observed decision at 200

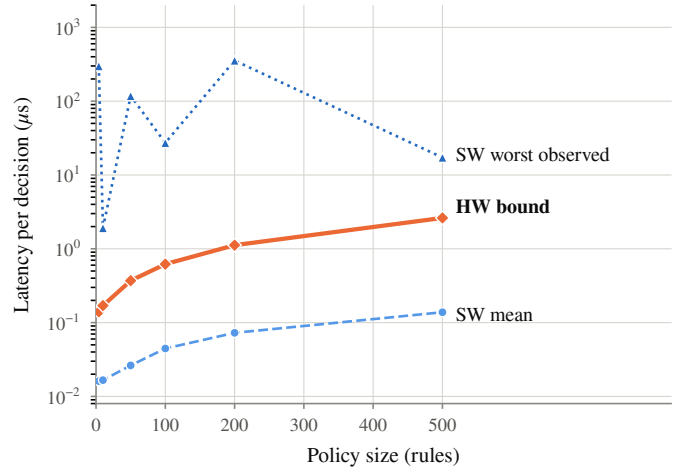


Fig. 4. Wall-clock latency per decision, log scale. The software mean and worst observed come from perf measurements of the five-dimensional path. The hardware line is the closed-form bound of (4) at 100 MHz, where minimum, mean and maximum coincide. The software worst case is an observation, not a bound, and does not track policy size.

rules was 355.4 μs against the fabric’s invariant 1.12 μs , and its worst at four rules was 298.8 μs against 0.14 μs . Further, the software figure is an observation, not a bound: the 157 μs outlier in the DCAS baseline arrived in a run whose mean was 1.14 μs [3]. An operating system does not guarantee its own tail, and a control loop budgets for the tail. Figure 4 plots both. The software mean runs below the hardware bound at every policy size and the worst case runs above it at every size, by more than three orders of magnitude at four rules. The worst case does not track policy size, because the tail belongs to the scheduler and not to the policy.

VIII. LIMITATIONS

First, the parity result covers three, four and five dimensions on one part at one clock. We have not shown that an arbitrary sixth dimension is free, only that these two were, and Section V names the mechanism that would end the pattern. Second, the software baseline is a 4.96 GHz laptop CPU while the fabric runs at 100 MHz. A baseline on the board’s own Cortex-A9 would make the wall-clock comparison fairer, and we did not run it. The board data bounds the fabric and the interface, not a software decision path on the same silicon. Third, software latency distributions were measured under WSL2, whose scheduler jitter contaminates maxima, which is why we cite worst observed values and never call them WCETs. Fourth, the primitive is capacity-bounded at 512 rules and 256 policy nodes, and policy update happens off the decision path. Fifth, the modeled external-state lookup is synthetic and excluded from every empirical claim, since the measured SQLite path shows the same thing. Sixth, the kernel compiles only the association subset of INCITS 565, so no claim extends beyond rules that grant. σ and π are likewise positive conditions: a rule requires a state or source type and cannot forbid one. Seventh, the zero-cycle result

holds at one operating point, 100 MHz on a Zynq-7020, where all three kernels close timing with +0.33 ns slack. At an aggressive clock target an added dimension could force another pipeline stage, the boundary the hypothesis of Section IV predicts. The two-to-three step in iteration latency already shows that mechanism operating below the threshold where it costs cycles. Characterizing the frequency ceiling is future work.

IX. CONCLUSION AND FUTURE WORK

We’ve determined that in hardware, security dimensionality is free in time. On a Zynq-7020 at 100 MHz, three-, four- and five-dimensional H-NGAC kernels resolve every decision in the same number of clock cycles at every policy size, yet across one 2,307-request corpus the three-dimensional kernel permits 1,443 requests the five-dimensional kernel denies. Latency is closed-form at $12 + n/2$ cycles, zero jitter by construction and confirmed on silicon. Area, not time, is what a dimension costs. Thus the security question a control path can afford to ask is no longer set by its latency budget. A robotics or functional-safety engineer fixes the authorization worst case at synthesis and spends the remaining margin on policy.

For future works, three questions remain outstanding: 1. At what clock does the next dimension stop being free, and does that ceiling move with policy width or only with stage depth? 2. What does a software decision path on the board’s own Cortex-A9 cost, so the wall-clock comparison runs at one class of silicon? 3. Does (4) survive a dynamic policy update path, or does update traffic reintroduce jitter? Further, we call on the community to report authorization performance as cycle-level worst cases rather than means, because a mean creates probabilistic uncertainty preventing tight control loop budgeting.

REFERENCES

- [1] National Vulnerability Database, “CVE-2022-33323: Mitsubishi electric MELFA unauthorized command execution,” <https://nvd.nist.gov/vuln/detail/CVE-2022-33323>, 2022.
- [2] —, “CVE-2021-38425: eProsima fast DDS RTPS message injection,” <https://nvd.nist.gov/vuln/detail/CVE-2021-38425>, 2021.
- [3] H. Karim, S. Sitharaman, and D. Gupta, “Hardware-accelerated NGAC authorization for real-time multi-robot systems,” in *2026 IEEE 19th Dallas Circuits and Systems Conference (DCAS)*, 2026, pp. 1–4.
- [4] H. Karim, D. Gupta, and S. Sitharaman, “Deterministic time-scoped NGAC for real-time multi-robot systems,” 2026, under review.
- [5] S. Sitharaman, H. Karim, D. Gupta, and M. Tyagi, “Scalable privilege analysis for multi-cloud big data platforms: A hypergraph approach,” in *2025 IEEE International Conference on Big Data (BigData)*, 2025, pp. 6626–6633.
- [6] InterNational Committee for Information Technology Standards, “Information technology — next generation access control (NGAC),” ANSI/INCITS, Tech. Rep. INCITS 565-2020, 2020.
- [7] D. Ferraiolo, V. Atluri, and S. Gavrila, “The Policy Machine: A novel architecture and framework for access control policy specification and enforcement,” *Journal of Systems Architecture*, vol. 57, no. 4, pp. 412–424, 2011.
- [8] P. Mell, J. Shook, R. Harang, and S. Gavrila, “Linear time algorithms to restrict insider access using multi-policy access control systems,” *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications (JoWUA)*, vol. 8, no. 1, 2017.
- [9] A. X. Liu, F. Chen, J. Hwang, and T. Xie, “Designing fast and scalable XACML policy evaluation engines,” *IEEE Transactions on Computers*, vol. 60, no. 12, pp. 1802–1817, 2011.
- [10] Open Policy Agent Project, “Open policy agent,” <https://www.openpolicyagent.org>, 2024, cNCF graduated project.
- [11] OASIS, “eXtensible access control markup language (XACML) version 3.0,” OASIS Standard, Tech. Rep., 2013.
- [12] T. Huffmire, S. Prasad, T. Sherwood, and R. Kastner, “Policy-driven memory protection for reconfigurable hardware,” in *Computer Security – ESORICS 2006, LNCS 4189*, 2006, pp. 461–478.
- [13] L. Fiorin, G. Palermo, S. Lukovic, V. Catalano, and C. Silvano, “Secure memory accesses on networks-on-chip,” *IEEE Transactions on Computers*, vol. 57, no. 9, pp. 1216–1229, 2008.
- [14] H. Song and J. W. Lockwood, “Efficient packet classification for network intrusion detection using FPGA,” in *Proc. ACM/SIGDA 13th Int. Symp. on Field-Programmable Gate Arrays (FPGA ’05)*, 2005, pp. 238–245.
- [15] T. Ganegedara, W. Jiang, and V. K. Prasanna, “A scalable and modular architecture for high-performance packet classification,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, pp. 1135–1144, 2014.
- [16] F. Restuccia, A. Meza, and R. Kastner, “Aker: A design and verification framework for safe and secure SoC access control,” in *Proc. IEEE/ACM Int. Conf. on Computer-Aided Design (ICCAD)*, 2021, pp. 1–9.
- [17] R. N. M. Watson, J. Woodruff, P. G. Neumann, S. W. Moore *et al.*, “CHERI: A hybrid capability-system architecture for scalable software compartmentalization,” in *2015 IEEE Symposium on Security and Privacy*, 2015.
- [18] V. Mayoral-Vilches, R. White, G. Caiazza, and M. Arguedas, “SROS2: Usable cyber security tools for ROS 2,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.
- [19] Object Management Group, “DDS security specification, version 1.1,” OMG, Tech. Rep., 2018.
- [20] B. Dieber, B. Breiling, S. Taurer, S. Kacianka, S. Rass, and P. Schartner, “Security for the robot operating system,” *Robotics and Autonomous Systems*, vol. 98, pp. 192–203, 2017.
- [21] S. Rivera, S. Lagraa, C. Nita-Rotaru, S. Becker, and R. State, “ROS-Defender: SDN-based security policy enforcement for robotic applications,” in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2019, pp. 114–119.
- [22] S. Salimi, F. Keramat, J. P. Queralta, and T. Westerlund, “A customizable conflict resolution and attribute-based access control framework for multi-robot systems,” *Journal of Systems Architecture*, vol. 168, p. 103528, 2025.
- [23] M. Hasan, S. Mohan, R. Pellizzoni, and R. B. Bobba, “Contego: An adaptive framework for integrating security tasks in real-time systems,” in *Proc. 29th Euromicro Conf. on Real-Time Systems (ECRTS), LIPIcs 76*, 2017, pp. 23:1–23:22.
- [24] M. Hasan, A. Kashinath, C.-Y. Chen, and S. Mohan, “SoK: Security in real-time systems,” *ACM Computing Surveys*, vol. 56, no. 9, pp. 218:1–218:31, 2024.
- [25] S. Pinisetty, P. S. Roop, S. Smyth, N. Allen, S. Tripakis, and R. von Hanxleden, “Runtime enforcement of cyber-physical systems,” *ACM Transactions on Embedded Computing Systems*, vol. 16, no. 5s, pp. 178:1–178:25, 2017.
- [26] D. Ferraiolo, R. Chandramouli, R. Kuhn, and V. Hu, “Extensible access control markup language (XACML) and next generation access control (NGAC),” in *Proc. 2016 ACM Int. Workshop on Attribute Based Access Control (ABAC ’16)*, 2016, pp. 13–24.
- [27] D. F. Ferraiolo, S. I. Gavrila, G. Katwala, and J. D. Roberts, “Imposing fine-grain next generation access control over database queries,” in *Proc. 2nd ACM Workshop on Attribute-Based Access Control (ABAC ’17)*, 2017.
- [28] National Vulnerability Database, “CVE-2022-45789: Schneider electric modicon session hijack,” <https://nvd.nist.gov/vuln/detail/CVE-2022-45789>, 2022.
- [29] G. Deng, G. Xu, Y. Zhou, T. Zhang, and Y. Liu, “On the (In)Security of secure ROS2,” in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2022, pp. 739–753.